

CODE EASY ACADEMY

JS MASTERY MARATHON · WEEK 04

Functions & Arrays

Write once. Use anywhere.

INSIDE THIS PACK

- 01 — **The 4 Function Concepts** · Declare, Call, Parameters, Return
- 02 — **void vs return** · The one distinction that matters
- 03 — **Arrays** · The Missing Piece
- 04 — **Live Demos** · 2 builds
- 05 — **Common Bugs** · 5 mistakes that will bite you
- 06 — **Exercise Pack** · 10 Problems
- 07 — **Self-check** · Before you submit

THE PROMISE THIS WEEK

By the end of this week, you stop writing the same code twice. You'll write a function once and call it anywhere. And you'll store multiple values in arrays instead of making 10 variables. These two things alone will change how you think about every program you write.

Submissions due Friday night.

PART 01 · THE 4 FUNCTION CONCEPTS

The core mental model. Everything else is just detail on top of these 4.

Declare	Define what the function does
Call	Actually run it
Parameters	Give it inputs so it's not the same every time
Return	Get a value back that your program can use

THE BIG IDEA

A function that isn't called never runs. **Declare** is the blueprint. **Call** is when it actually happens.

Concept 1 · Declare and Call

```
// Declare -- define the function
function sayHello() {
  console.log("Hello!");
}

// Call -- actually run it
sayHello(); // Hello!
sayHello(); // Hello!
sayHello(); // Hello!
```

Write once, run anywhere, run as many times as you want. That's the entire point.

Concept 2 · Parameters

Parameters are the inputs you hand to a function. Without them, every function does the exact same thing every time — useless.

```
function greet(name) {
  console.log("Hello, " + name + "!");
}

greet("Rajdeep"); // Hello, Rajdeep!
greet("Priya");   // Hello, Priya!
greet("Arjun");   // Hello, Arjun!
```

One function. Three different outputs. That's the power.
Multiple parameters — separate them with commas:

```
function add(a, b) {
  console.log(a + b);
}

add(3, 5); // 8
```

Concept 3 · Return

`console.log` SHOWS a value. `return` GIVES it back so your program can use it.

```
// This only prints -- the value disappears after
function addAndPrint(a, b) {
  console.log(a + b);
}

// This returns -- the value is yours to use
function add(a, b) {
  return a + b;
}

let result = add(3, 5);
console.log(result);           // 8
console.log(result * 2);       // 16
console.log("Sum is " + result); // Sum is 8
```

THE RULE: If you need the value for something else — use `return`. If you just want to show it once — `console.log` is fine. Most real functions return.

Concept 4 · void vs return

```
// void -- does something, gives nothing back
function showWelcome(name) {
  console.log("Welcome, " + name);
}

// return -- gives something back
function getWelcome(name) {
  return "Welcome, " + name;
}

// void: you can't use the result
let msg = showWelcome("Priya"); // msg is undefined

// return: you can
let msg2 = getWelcome("Priya"); // msg2 = "Welcome, Priya"
```

ASK YOURSELF BEFORE WRITING ANY FUNCTION: "Does the caller need a value back?"

Yes → `return`. No → just `console.log` inside.

PART 02 · ARRAYS: THE MISSING PIECE

Until now you've been doing this:

```
let student1 = "Aman";
let student2 = "Priya";
let student3 = "Rohit";
// ... 30 students = 30 variables. Disaster.
```

Arrays fix this:

```
let students = ["Aman", "Priya", "Rohit", "Sneha", "Karan"];
```

One variable. Any number of items.

The 4 Things You Need To Know About Arrays

1 · Access by index (starts at 0)

```
let fruits = ["apple", "mango", "banana"];
console.log(fruits[0]); // apple
console.log(fruits[1]); // mango
console.log(fruits[2]); // banana
```

2 · .length — how many items

```
console.log(fruits.length); // 3
```

3 · Loop through an array

```
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

4 · Add and remove items

```
fruits.push("grapes"); // add to end
fruits.pop();           // remove from end
```

Why Functions + Arrays Belong Together

This is where Week 3 and Week 4 connect. Remember the Password Checker? You built it in 40 lines. Now watch what happens when you wrap it in a function:

```
// Before -- everything tangled together
let hasDigit = false;
for (let i = 0; i < password.length; i++) {
  if (password[i] >= "0" && password[i] <= "9") {
    hasDigit = true;
  }
}

// After -- clean, reusable, readable
function hasDigit(password) {
  for (let i = 0; i < password.length; i++) {
    if (password[i] >= "0" && password[i] <= "9") {
      return true;
    }
  }
  return false;
}
```

Same logic. But now you can call `hasDigit(password)` anywhere — in 10 different projects — without rewriting it.

PART 03 · LIVE DEMOS

Demo 1 · Greeter Function

Simple intro — declare, parameters, return

```
function greet(name, timeOfDay) {  
  return "Good " + timeOfDay + ", " + name + "!";  
}  
  
console.log(greet("Aman", "morning")); // Good morning, Aman!  
console.log(greet("Priya", "evening")); // Good evening, Priya!
```

WHAT TO NOTICE: Same function, different inputs, different outputs. This is reusability in its simplest form.

Demo 2 · Password Checker Refactored

Week 3's Star Problem — now clean. Last week this was 40+ lines tangled in a `while` loop. Watch what functions do to it:

```

function hasDigit(password) {
  for (let i = 0; i < password.length; i++) {
    if (password[i] >= "0" && password[i] <= "9") return true;
  }
  return false;
}

function hasUpper(password) {
  for (let i = 0; i < password.length; i++) {
    if (password[i] >= "A" && password[i] <= "Z") return true;
  }
  return false;
}

function isStrongPassword(password) {
  let reasons = [];
  if (password.length < 8) reasons.push("must be at least 8 characters");
  if (!hasDigit(password)) reasons.push("must contain a digit");
  if (!hasUpper(password)) reasons.push("must contain an uppercase letter");
  return reasons;
}

let attemptsLeft = 3;
while (attemptsLeft > 0) {
  let password = prompt("Enter a strong password:");
  let reasons = isStrongPassword(password);
  if (reasons.length === 0) {
    console.log("Password set!");
    break;
  } else {
    console.log("Weak: " + reasons.join(", "));
    attemptsLeft--;
    console.log("Attempts left: " + attemptsLeft);
  }
}

```

NOTICE WHAT HAPPENED: The `while` loop is now only 10 lines. Each rule is its own function. You can test `hasDigit` by itself. You can reuse `isStrongPassword` in any project. This is what professional code looks like.

PART 04 · COMMON BUGS

The 5 mistakes that will eat your time.

Bug 1 · Declaring but never calling

```
function greet() {  
  console.log("Hello!"); // This never runs  
}  
// You forgot: greet();
```

Bug 2 · Missing return (getting undefined)

```
function add(a, b) {  
  let result = a + b;  
  // forgot: return result  
}  
let x = add(3, 4);  
console.log(x); // undefined -- not 7
```

Bug 3 · Return stops the function — dead code

```
function check(n) {  
  return "done";  
  console.log(n); // DEAD CODE -- never runs  
}
```

Everything after `return` is ignored. Use it intentionally.

Bug 4 · Off-by-one in arrays

```
let scores = [90, 85, 78];  
console.log(scores[3]); // undefined -- last index is 2, not 3
```

Arrays start at 0. Last index is always `length - 1`.

Bug 5 · Scope — variable only exists inside the function

```
function calculate() {  
  let result = 42;  
}  
console.log(result); // ReferenceError -- result doesn't exist out here
```

Variables declared inside a function stay inside that function. This is called **scope**. It's a feature, not a bug — but it will confuse you until it clicks.

PART 05 · EXERCISE PACK

10 problems · 2 Easy · 5 Medium · 3 Challenge · + Bonus

HOW TO TACKLE EACH PROBLEM

1. **Decompose** — Inputs, Process, Output, Edges in plain English
2. **Name the pattern** — what does this function do? what does it return?
3. **Pseudocode** — plain English steps first
4. **JavaScript** — only now open the editor

PROBLEM 01 ★ EASY · Square and Cube

Write two functions: `square(n)` and `cube(n)`. Each takes a number and returns the result. Then write a third function `printPowers(n)` that calls both and prints:

Square of 4: 16

Cube of 4: 64

Edges: $n = 0$, $n = \text{negative}$, $n = \text{decimal}$

PROBLEM 02 ★ EASY · Array Sum

Write a function `sumArray(numbers)` that takes an array of numbers and returns their total.

Example: `sumArray([10, 20, 30])` → 60

Edges: empty array (return 0), array with one item, negative numbers

PROBLEM 03 ★★ MEDIUM · Find Maximum

Write a function `findMax(numbers)` that takes an array and returns the largest number. Don't use `Math.max()`.

Example: `findMax([3, 9, 1, 7, 5])` → 9

Edges: all same values, one item, all negatives

PROBLEM 04 ★★ MEDIUM · Count Occurrences

Write a function `countOccurrences(arr, target)` that counts how many times `target` appears in `arr`.

Example: `countOccurrences(["a", "b", "a", "c", "a"], "a")` → 3

Edges: target not in array (return 0), empty array, numbers vs strings

PROBLEM 05 ★★ MEDIUM · Grade Classifier

Write a function `getGrade(score)` that returns a grade:

90+ → "A" | 75-89 → "B" | 60-74 → "C" | Below 60 → "Fail"

Then write `printClassReport(names, scores)` that takes two arrays and prints each student's name + grade.

Edges: score exactly 90, exactly 60, score above 100, negative score

PROBLEM 06 ★★ MEDIUM · Reverse an Array

Write a function `reverseArray(arr)` that returns a new reversed array. Don't use `.reverse()`.

Example: `reverseArray([1, 2, 3, 4, 5])` → [5, 4, 3, 2, 1]

Edges: empty array, single item, already reversed

PROBLEM 07 ★★ MEDIUM · Remove Duplicates

Write a function `removeDuplicates(arr)` that returns a new array with duplicates removed. Order should be preserved.

Example: `removeDuplicates([1, 2, 2, 3, 1, 4]) → [1, 2, 3, 4]`

Hint: Use a `results` array. For each item, only push it if it isn't already in `results`.

Edges: all duplicates, no duplicates, single item

PROBLEM 08 ★★★ CHALLENGE · Student Report Card

You have two arrays: `names = ["Aman", "Priya", "Rohit"]` and `scores = [88, 45, 73]`.

Write a function `generateReport(names, scores)` that:

- Prints each student's name, score, and grade (use `getGrade` from Problem 5)
- Prints the class average
- Prints who scored highest

Edges: all failing, all same score, one student

PROBLEM 09 ★★★ CHALLENGE · STAR PROBLEM

Cart Total with Discounts

WHY THIS IS THE STAR: It chains multiple functions together — the same way real apps work. Get this right and you'll think like a developer, not just a coder.

Write these functions:

- `calculateSubtotal(prices)` — sum of all prices
- `applyDiscount(subtotal, discountPercent)` — returns discounted amount
- `applyTax(amount, taxRate)` — returns amount with tax
- `printReceipt(items, prices, discountPercent, taxRate)` — calls all the above and prints a clean receipt

Example output:

--- RECEIPT ---

Shoes: Rs.2000

T-Shirt: Rs.800

Subtotal: Rs.2800

Discount (10%): -Rs.280

After discount: Rs.2520

Tax (18%): +Rs.453.60

Total: Rs.2973.60

Edges: 0% discount, 0% tax, empty cart

HINT: Draw the 4 functions and how they connect BEFORE you open the editor. Each function takes the output of the previous one. That's function chaining — and it's how every real app is built.

PROBLEM 10 ★★★ CHALLENGE · Find and Filter

Write two functions:

- `findFirst(arr, target)` — returns the index of the first match, or -1 if not found

- `filterAbove(numbers, threshold)` — returns a new array with only numbers above the threshold

Examples:

`findFirst(["a","b","c","b"], "b") → 1`

`filterAbove([10, 25, 8, 40, 15], 20) → [25, 40]`

Edges: target not found, empty array, all items above threshold, none above threshold

BONUS CHALLENGE · Playlist Analyser (revisited)

Remember the Bonus from Week 3? Rewrite it using functions.

Songs: [210, 180, 240, 195, 220, 200, 255]

Write:

- `getTotalDuration(songs)` → returns total seconds
- `formatDuration(seconds)` → returns "24:40" format
- `findLongest(songs)` → returns duration of longest song
- `countOver(songs, threshold)` → returns count of songs above threshold

Then call all four and print a clean summary.

THE POINT: Compare this to your Week 3 solution. Same problem — but now each piece is testable, reusable, and readable. That's the difference functions make.

PART 06 · SELF-CHECK BEFORE YOU SUBMIT

- ☐ Did I write functions that `return` values instead of just `console.log`-ing inside?
- ☐ Did I test each function with at least 2 different inputs?
- ☐ Did I handle the edges listed in each problem?
- ☐ For the Star Problem — did I chain functions instead of writing everything in one block?
- ☐ Can I explain what each function does in one sentence?

IF YOU ONLY DO ONE THING

Do Problem 9. It's the one where functions stop being a concept and start being a tool. Skip the easy ones if you must. Do not skip the Star.

Post your code to the WhatsApp group by Friday night. We solve them together at Logic Lab.

Stuck on Problem 7 or 8? The hint is in your Week 3 notes — same loop patterns, just inside functions now.

See you Friday for Logic Lab.

— Rajdeep